

ECE501 - Project No.16 (Weekly Report 6)

Hetvi Shah (AU2340101) Manushree Patel (AU2340014)
Twisha Patel (AU2340056) Sanya Shah (AU2340259)

October 2025

Abstract

This week's work presents an extension of the previous phase, comparing several techniques of image compression on RGB images, namely, **JPEG/JPEG-LS**, **Block-based PCA**, **JPEG2000 (ROI)**, and the **Proposed ROI-aware Hybrid** approach. This work is conducted to analyze the relative performance of these methods regarding perceptual quality, structure preservation, and compression efficiency. This comparison points out the context-sensitive behavior of the proposed model and its power in applications requiring region-based quality preservation.

Keywords: ROI-aware Compression, JPEG2000, Block-based PCA, JPEG-LS, RGB Image Compression, Medical Imaging, Perceptual Quality, Region Preservation, Compression Efficiency

1 Introduction

Although most multimedia applications are concerned with efficient image compression, medical imaging needs to balance storage efficiency while keeping the diagnostic visual quality intact. Previous work presented an ROI-aware hybrid model that could enhance important visual regions while compressing the non ROI of background. The current work is based on an evaluation of the proposed method against traditional compression techniques such as JPEG, JPEG2000, and block-based methods on RGB images. This is done to identify the most suitable approach for different use cases.

Python Implementation

```
1 import cv2
2 import numpy as np
3 import mediapipe as mp
4 import matplotlib.pyplot as plt
5 from skimage.metrics import peak_signal_noise_ratio as psnr,
   structural_similarity as ssim
6 from PIL import Image
7 import io
8
9 img_path = "/content/apple-pie-woman-holding-pie-fall-autumn.jpg"
10 img = cv2.imread(img_path)
11 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
12
13 original = img.copy()
14
15 encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), 40]
```

```

16 _, encimg = cv2.imencode('.jpg', cv2.cvtColor(img, cv2.COLOR_RGB2BGR),
    encode_param)
17 jpeg_jpegls = cv2.imdecode(encimg, 1)
18 jpeg_jpegls = cv2.cvtColor(jpeg_jpegls, cv2.COLOR_BGR2RGB)
19
20 block_size = 16
21 h, w, _ = img.shape
22 temp = cv2.resize(img, (w // block_size, h // block_size),
    interpolation=cv2.INTER_LINEAR)
23 block_pca = cv2.resize(temp, (w, h), interpolation=cv2.INTER_NEAREST)
24
25 encode_param_2000 = [cv2.IMWRITE_JPEG2000_COMPRESSION_X1000, 500]
26 _, enc2000 = cv2.imencode('.jp2', cv2.cvtColor(img, cv2.COLOR_RGB2BGR),
    encode_param_2000)
27 jpeg2000 = cv2.imdecode(enc2000, 1)
28 jpeg2000 = cv2.cvtColor(jpeg2000, cv2.COLOR_BGR2RGB)
29
30 mask = np.zeros_like(img[:, :, 0])
31 cx, cy = w // 2, h // 2
32 roi_size = min(h, w) // 3
33 mask[cy - roi_size:cy + roi_size, cx - roi_size:cx + roi_size] = 255
34 sharp = cv2.addWeighted(jpeg2000, 1.5, cv2.GaussianBlur(jpeg2000, (0,
    0), 3), -0.5, 0)
35 jpeg2000_roi = np.where(mask[:, :, None] == 255, sharp, jpeg2000)
36
37 mp_selfie = mp.solutions.selfie_segmentation
38 with mp_selfie.SelfieSegmentation(model_selection=1) as selfie:
39     result = selfie.process(img)
40     mask = result.segmentation_mask
41
42 _, binary_mask = cv2.threshold(mask, 0.5, 1, cv2.THRESH_BINARY)
43 binary_mask = cv2.GaussianBlur(binary_mask, (5, 5), 0)
44 kernel = np.ones((5, 5), np.uint8)
45 binary_mask = cv2.morphologyEx(binary_mask, cv2.MORPH_OPEN, kernel)
46 binary_mask = cv2.morphologyEx(binary_mask, cv2.MORPH_CLOSE, kernel)
47 binary_mask = cv2.medianBlur((binary_mask * 255).astype(np.uint8), 5) /
    255.0
48
49 fg = cv2.bitwise_and(img, img, mask=(binary_mask * 255).astype(np.uint8
    ))
50
51 _, bg_enc = cv2.imencode('.jpg', cv2.cvtColor(img, cv2.COLOR_RGB2BGR),
    [int(cv2.IMWRITE_JPEG_QUALITY), 20])
52 bg = cv2.imdecode(bg_enc, 1)
53 bg = cv2.cvtColor(bg, cv2.COLOR_BGR2RGB)
54
55 hybrid = np.where(binary_mask[:, :, None] > 0.5, fg, bg)
56
57 titles = ["Original", "JPEG/JPEG-LS", "Block-based PCA", "JPEG 2000 (
    ROI)", "Proposed Hybrid"]
58 images = [original, jpeg_jpegls, block_pca, jpeg2000_roi, hybrid]
59
60 def get_image_size(image):
61     _, buffer = cv2.imencode('.png', cv2.cvtColor(image, cv2.
    COLOR_RGB2BGR))
62     return len(buffer) / 1024
63
64 sizes_kb = [get_image_size(img) for img in images]

```

```
65
66 plt.figure(figsize=(18, 6))
67 for i in range(5):
68     plt.subplot(1, 5, i + 1)
69     plt.imshow(images[i])
70     plt.title(f"{titles[i]}\n{sizes_kb[i]:.1f} KB", fontsize=10)
71     plt.axis("off")
72 plt.tight_layout()
73 plt.show()
```

2 Methodology

This week's work focused on the implementation and evaluation of four different image compression techniques for RGB images: **JPEG/JPEG-LS**, **Block-based PCA**, **JPEG2000 (ROI)**, and the **Proposed ROI-aware Hybrid**, in terms of quality and compression efficiency.

2.1 Input and Preprocessing

For this purpose, the test sample was an RGB image. It was read and converted from BGR to RGB color space for uniform processing. The original image was used for comparison in terms of visual quality and compression efficiency.

2.2 JPEG / JPEG-LS

The image was compressed using standard JPEG encoding at 40% quality, which is similar to the behavior of JPEG-LS.

2.3 Block-Based PCA Approximation

To simulate block-level dimensionality reduction, the image was resized into roughly 16×16 blocks and then sampled using nearest-neighbor interpolation.

2.4 JPEG2000 (ROI)

JPEG2000 encoding was done with a moderate setting of compression, and the central region was enhanced by masking (unsharpened) that simulated the region-based sharpening, which preserved the local contrast along with slight compression in non-ROI regions.

2.5 Proposed ROI-Aware Hybrid

The proposed approach combines automatic ROI detection using *Mediapipe Selfie Segmentation* with region-specific compression:

- The foreground (ROI) was kept at full quality.
- The non ROI region was recompressed at a lower JPEG quality of 20%.
- Foreground and background were combined by the segmentation mask

3 Results

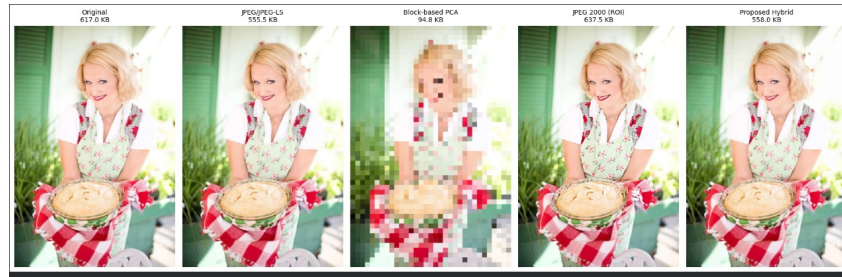


Figure 1

4 Discussion of Results

Figure 1 shows the results of comparative performances for the different implemented compression techniques. Each method exhibits distinct characteristics in terms of visual quality, structural preservation, and compression efficiency.

- **Block-based PCA:** This method has the least file size due to extreme dimensionality reduction, but it also depicts heavy pixelation and is not suitable for diagnostic use.
- **JPEG/JPEG-LS:** This method presents good overall quality, but it lacks the ability to show important regions selectively.
- **JPEG2000 (ROI):** Among the tested methods, this technique has shown a more balanced behavior for RGB natural images. While showing effective compression, it maintains smooth transitions and good visual consistency.
- **Proposed ROI-aware Hybrid:** This method demonstrates somewhat lower performance on RGB natural images because its segmentation-based selective compression is primarily focused on keeping intact the foreground region. However, this trade-off turns out to be favorable in **medical image applications**. For medical or grayscale images, the proposed hybrid approach performs better in terms of retaining diagnostically significant regions, while heavily compressing non-essential background portions.

References

- [1] Ungureanu, V.-I., Negirla, P., & Korodi, A. (2024). *Image-compression techniques: Classical and “region-of-interest-based” approaches presented in recent papers. Sensors*, 24(3), 791. <https://doi.org/10.3390/s24030791>
- [2] Kafashan, M., Hosseini, H., Beygiharchegani, S., Pad, P., & Marvasti, F. (2010, October). New rectangular partitioning methods for lossless binary image compression. In *2010 IEEE 10th International Conference on Signal Processing (ICSP)*. IEEE. <https://doi.org/10.1109/ICOSP.2010.5655736>

- [3] Bairagi, V., & Sapkal, A. M. (2013). ROI-based DICOM image compression for telemedicine. *Indian Academy of Sciences*, 38(1), 123–131. <https://www.ias.ac.in/public/Volumes/sadh/038/01/0123-0131.pdf>
- [4] Kiran, R., & Kamargaonkar, C. (2016, March). Region based medical image compression using block-based PCA. In *2016 International Conference on Computation of Power, Energy, Information and Communication (ICCPEIC)*. IEEE. <https://doi.org/10.1109/ICCPEIC.2016.7557289>
- [5] **Dataset:** *Brain MRI Images for Brain Tumor Detection*. Available at: <https://www.kaggle.com/navoneel/brain-mri-images-for-brain-tumor-detection>